

1. ¿Cuál es la diferencia entre monitoreo, observabilidad y alertamiento?

El **monitoreo** consiste en recolectar métricas predefinidas de un sistema para saber si está funcionando dentro de parámetros normales. La **observabilidad** va más allá: permite entender el comportamiento interno del sistema a partir de sus salidas (métricas, logs y trazas), incluso ante situaciones no previstas. El **alertamiento** es el mecanismo que notifica a los operadores cuando alguna métrica supera un umbral crítico, permitiendo reaccionar ante problemas.

2. ¿Por qué Prometheus trabaja bajo un modelo de scraping?

Prometheus utiliza un modelo **pull** (scraping), donde él mismo consulta periódicamente los endpoints de métricas de cada servicio. Esto centraliza el control en Prometheus, facilita la detección de servicios caídos (si no responde al scrape, está down), y evita que cada servicio necesite saber a dónde enviar sus métricas.

3. ¿Qué es un exporter y por qué se necesita?

Un **exporter** es un programa que traduce las métricas de un servicio existente al formato que Prometheus puede consumir. Se necesita porque muchos servicios (como MariaDB, Tomcat o el sistema operativo) no exponen métricas en formato Prometheus de forma nativa. El exporter actúa como adaptador entre el servicio y Prometheus.

4. ¿Qué diferencia hay entre monitorear el host y monitorear contenedores?

Monitorear el **host** significa observar los recursos físicos o virtuales del servidor: CPU, RAM, disco y red a nivel de sistema operativo, usando herramientas como node_exporter. Monitorear **contenedores** implica observar el consumo de recursos por cada contenedor individualmente, su estado, reinicios y uso de red, usando herramientas como cAdvisor que entienden la abstracción de Docker.

5. ¿Qué métricas son más importantes para detectar saturación del servidor?

Las métricas más críticas son: uso de CPU (especialmente si supera el 80% sostenido), uso de memoria RAM (por encima del 85% indica riesgo de OOM), uso de disco (un disco lleno puede tumbar todos los servicios), carga del sistema (load average) y tráfico de red. Estas métricas en conjunto permiten anticipar cuellos de botella antes de que afecten a los usuarios.

6. ¿Qué métricas son relevantes para una aplicación Django?

Para Django las métricas más importantes son: número de requests HTTP por minuto, códigos de respuesta HTTP (especialmente errores 4xx y 5xx), latencia de requests, tiempo de consultas a la base de datos, número de conexiones activas y disponibilidad del endpoint /metrics. Estas métricas permiten detectar degradación en el rendimiento o fallos en la aplicación.

7. ¿Qué métricas son relevantes para Tomcat/JVM?

Para Tomcat y la JVM las métricas clave son: uso de memoria heap y non-heap, actividad del Garbage Collector (frecuencia y duración), número de threads activos, requests procesados por segundo, errores HTTP, tiempo de respuesta y estado general del servicio. Un heap lleno o un GC muy frecuente son señales claras de problemas de memoria en la aplicación Java.

8. ¿Qué riesgos existen al exponer Prometheus, Grafana o exporters públicamente?

Exponer estos servicios públicamente representa varios riesgos: un atacante podría obtener información detallada sobre la infraestructura (versiones, IPs, recursos), modificar dashboards o alertas, deshabilitar el monitoreo para pasar desapercibido durante un ataque, o usar los exporters como vector de ataque. Por eso en esta práctica Prometheus y los exporters se mantienen solo en red interna y solo Grafana se expone mediante proxy inverso con HTTPS.

9. ¿Por qué no deben guardarse contraseñas SMTP en el repositorio?

Porque los repositorios, incluso los privados, pueden ser accedidos por múltiples personas o quedar expuestos accidentalmente. Una contraseña SMTP en el repositorio permite a cualquier persona enviar correos desde esa cuenta, lo que puede usarse para spam o phishing. La práctica correcta es usar variables de entorno o archivos .env excluidos del repositorio mediante .gitignore, como se hizo en esta práctica.

10. ¿Qué mejoras implementarían si esta solución se llevara a producción?

Para producción se implementarían las siguientes mejoras: usar un archivo .env para todas las credenciales sensibles, agregar autenticación en Prometheus mediante reverse proxy, implementar retención de datos configurada en Prometheus con almacenamiento persistente externo (como S3), usar Grafana con base de datos PostgreSQL en lugar de SQLite, agregar logs centralizados con Loki, configurar alta disponibilidad para Prometheus con Thanos o Cortex, y establecer un pipeline CI/CD para desplegar cambios de configuración automáticamente.